Java Fullstack + Angular E-Commerce Build Guide

SportsCenter - A step-by-step guide to building a production-ready fullstack e-commerce app from scratch

Spring Boot 3 | Java 21 | MySQL | Redis | JWT | Angular 16 | Docker

Table of Contents

STEP 0 - Project Overview and Architecture

SportsCenter is a fullstack e-commerce app for sports equipment. It uses a Spring Boot REST API backend with Angular 16 SPA frontend. This guide walks through building every layer from scratch, explaining the **why** behind each architectural decision.

What the app does

- Browse a paginated product catalog filtered by Brand and Type
- Search products by keyword with live results
- Add/remove items from a shopping basket (persisted in Redis)
- User login with JWT authentication and protected routes in Angular
- Checkout flow with address and shipment selection

Tech Stack

Layer	Technology	Purpose
Backend API	Spring Boot 3.2 (Java 21)	REST endpoints, business logic
Relational DB	MySQL 8	Product catalog, brands, types
Cache / Basket	Redis 7	Shopping basket (fast key-value store)
Security	Spring Security + JWT	Stateless auth - no server sessions
Frontend	Angular 16	SPA with lazy-loaded feature modules
UI	Bootstrap 5 / Bootswatch	Responsive styling
State Mgmt	RxJS BehaviorSubject	Reactive basket and user state
Containers	Docker Compose	Local MySQL + Redis in one command

Architecture Flow

```
Angular (port 4200) --HTTP--&gt; Spring Boot API (port 8080) --&gt; MySQL
(products/brands/types)
--&gt; Redis (basket data)
```

STEP 1 - Prerequisites and Tool Installation

Install all tools before writing any code:

Tool	Version	Download / Install
Java JDK	21 (LTS)	https://adoptium.net
Apache Maven	3.9+	https://maven.apache.org (or use mvnw wrapper)
Node.js	18+ LTS	https://nodejs.org
Angular CLI	16	npm install -g @angular/cli@16
Docker Desktop	Latest	https://www.docker.com/products/docker-desktop
IntelliJ IDEA	Community	https://www.jetbrains.com/idea
VS Code	Latest	https://code.visualstudio.com (for Angular)
Postman	Latest	https://www.postman.com (API testing)

Verify installations

```
java -version # Should print: openjdk 21...
mvn -version # Should print: Apache Maven 3.9...
node -v # Should print: v18... or v20...
ng version # Should print: Angular CLI: 16...
docker --version # Should print: Docker version 24...
```

STEP 2 - Docker Setup - MySQL + Redis

Start MySQL and Redis locally with Docker Compose before writing any Java code. This avoids installing them manually and ensures a reproducible environment.

Create docker/docker-compose.yml

```
version: '3.1'
services:
ecommerce-mysql:
image: mysql
restart: always
environment:
MYSQL_ROOT_PASSWORD: 'password'
ports:
- '3306:3306'
volumes:
- ./data.sql:/docker-entrypoint-initdb.d/data.sql
- mysql-data:/var/lib/mysql

ecommerce-redis:
image: 'redis:latest'
container_name: 'my-redis-container'
ports:
- '6379:6379'
volumes:
- redis-data:/data

volumes:
redis-data:
mysql-data:
```

Note: Place data.sql in the docker/ folder. Docker auto-runs it to create and seed the sportscenter database on first startup.

Start the containers

```
cd docker/
docker compose up -d
docker ps # verify both containers are Up
```

STEP 3 - Spring Boot Project Setup

Go to https://start.spring.io and generate a project with these settings:

- Project: Maven | Language: Java | Spring Boot: 3.2.x
- Group: com.ecoomerce | Artifact: sportscenter
- Java version: 21
- Dependencies: Spring Web, Spring Data JPA, Spring Data Redis, Spring Security, MySQL Driver, Lombok

Add JWT dependencies to pom.xml

```
&lt;!-- JWT API (compile-time) --&gt;
&lt;dependency&gt;
&lt;groupId&gt;io.jsonwebtoken&lt;/groupId&gt;
&lt;artifactId&gt;jjwt-api&lt;/artifactId&gt;
&lt;version&gt;0.11.5&lt;/version&gt;
&lt;/dependency&gt;
&lt;dependency&gt;
&lt;groupId&gt;io.jsonwebtoken&lt;/groupId&gt;
&lt;artifactId&gt;jjwt-impl&lt;/artifactId&gt;
&lt;version&gt;0.11.5&lt;/version&gt;
&lt;scope&gt;runtime&lt;/scope&gt;
&lt;/dependency&gt;
&lt;dependency&gt;
&lt;groupId&gt;io.jsonwebtoken&lt;/groupId&gt;
&lt;artifactId&gt;jjwt-jackson&lt;/artifactId&gt;
&lt;version&gt;0.11.5&lt;/version&gt;
&lt;scope&gt;runtime&lt;/scope&gt;
&lt;/dependency&gt;
```

Configure src/main/resources/application.yaml

```
server:
port: 8080

spring:
datasource:
url: jdbc:mysql://localhost:3306/sportscenter
username: root
password: password
jpa:
hibernate:
ddl-auto: update
properties:
hibernate:
dialect: org.hibernate.dialect.MySQL8Dialect
data:
redis:
host: localhost
port: 6379
```

Note: ddl-auto: update lets Hibernate auto-create tables from your entity classes. In production use 'validate' and manage schema with Flyway or Liquibase.

STEP 4 - Database Entities - Product, Brand, Type (JPA / MySQL)

JPA entities map Java classes to MySQL tables. Brand and Type are lookup tables. Product is the main catalog table with foreign keys to both.

Brand.java

```
@Entity @Table(name="Brand")
@Data @AllArgsConstructor @NoArgsConstructor @Builder
public class Brand {
@Id @GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name="Id") private Integer id;
@Column(name="Name") private String name;

// One brand -> many products (back-reference, lazy loaded)
@OneToMany(mappedBy = "brand", fetch = FetchType.LAZY)
private List&lt;Product&gt; products;
}
```

Type.java (identical structure to Brand)

```
@Entity @Table(name="Type") // Fields: id, name, List&lt;Product&gt; products
```

Product.java (owns both foreign keys)

```
@Entity @Table(name="Product")
@Data @AllArgsConstructor @NoArgsConstructor @Builder
public class Product {
@Id @GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name="Id") private Integer id;
@Column(name="Name") private String name;
@Column(name="Description") private String description;
@Column(name="Price") private Long price;
@Column(name="PictureUrl") private String pictureUrl;

// Many products share one Brand - FK: ProductBrandId
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name="ProductBrandId", referencedColumnName="Id")
private Brand brand;

// Many products share one Type - FK: ProductTypeId
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name="ProductTypeId", referencedColumnName="Id")
private Type type;
}
```

Note: FetchType.LAZY means Brand and Type are NOT loaded until you call product.getBrand() - avoids costly JOIN queries when you only need the product list.

STEP 5 - Redis Entities - Basket and BasketItem

The basket is stored in **Redis** not MySQL. Redis is chosen because baskets are temporary, write-heavy, and fit naturally as a key-value document.

Basket.java

```
@Data @NoArgsConstructor
@RedisHash("Basket") // Stored as Redis key: Basket:&lt;id&gt;
public class Basket {
@Id // Redis @Id (String, not Integer)
private String id; // UUID/CUID generated by the Angular client
private List&lt;BasketItem&gt; items = new ArrayList&lt;&gt;();
public Basket(String id) { this.id = id; }
}
```

BasketItem.java

```
@Data @RedisHash("BasketItem")
public class BasketItem {
@Id private Integer id; // product ID
private String name; // snapshot of name at time of add
private String description;
private Long price; // snapshot price (pricing integrity)
private String pictureUrl;
private String productBrand; // denormalised - avoids MySQL join
private String productType; // denormalised - avoids MySQL join
private Integer quantity;
}
```

Note: BasketItem is denormalised - it stores brand/type as strings instead of IDs. This preserves what the user saw even if the catalog changes later.

STEP 6 - Spring Data Repositories

Repositories provide database operations without writing SQL. Define an interface; Spring generates the implementation at startup.

ProductRepository.java - JPA (MySQL)

```
@Repository
public interface ProductRepository extends JpaRepository&lt;Product, Integer&gt; {
// Free methods: findById, findAll(Pageable), save, deleteById

@Query("SELECT p FROM Product p WHERE p.name LIKE %:keyword%")
List&lt;Product&gt; searchByName(@Param("keyword") String keyword);

@Query("SELECT p FROM Product p WHERE p.brand.id = :brandId")
List&lt;Product&gt; searchByBrand(@Param("brandId") Integer brandId);

@Query("SELECT p FROM Product p WHERE p.type.id = :typeId")
List&lt;Product&gt; searchByType(@Param("typeId") Integer typeId);

@Query("SELECT p FROM Product p WHERE p.brand.id=:brandId AND p.type.id=:typeId")
List&lt;Product&gt; searchByBrandAndType(@Param("brandId") Integer b, @Param("typeId")
Integer t);

@Query("SELECT p FROM Product p WHERE p.brand.id=:brandId AND p.type.id=:typeId AND p.name
LIKE %:keyword%")
List&lt;Product&gt; searchByBrandTypeAndName(@Param("brandId") Integer b,
@Param("typeId") Integer t,
@Param("keyword") String k);
}
```

BasketRepository.java - Redis

```
@Repository
// CrudRepository (NOT JpaRepository) - Spring Data Redis implementation
public interface BasketRepository extends CrudRepository&lt;Basket, String&gt; {
// findById, save, deleteById, findAll all provided automatically
}
```

STEP 7 - DTO Response Models

DTOs (Data Transfer Objects) are what the API actually returns to clients - never expose JPA entities directly. This prevents infinite recursion from bidirectional relationships and lets you control exactly what data is sent.

```
// ProductResponse.java - what /api/products returns
@Data @Builder @NoArgsConstructor @AllArgsConstructor
public class ProductResponse {
private Integer id;
private String name;
private String description;
private Long price;
private String pictureUrl;
private String productType; // String, not Type entity
private String productBrand; // String, not Brand entity
}

// BrandResponse.java
@Data @Builder @NoArgsConstructor @AllArgsConstructor
public class BrandResponse { private Integer id; private String name; }

// TypeResponse.java (same as BrandResponse)

// BasketResponse.java
@Data @Builder @NoArgsConstructor @AllArgsConstructor
public class BasketResponse {
private String id;
private List&lt;BasketItemResponse&gt; items;
}

// JwtRequest.java - login payload from client
@Data @AllArgsConstructor @NoArgsConstructor
public class JwtRequest { private String username; private String password; }

// JwtResponse.java - login response to client
@Data @Builder
public class JwtResponse { private String username; private String token; }

// CustomErrorResponse.java - error payload
@Data @AllArgsConstructor
public class CustomErrorResponse {
private HttpStatus status; private String message; private String detail;
}
```

STEP 8 - Service Layer - Business Logic

Services sit between controllers and repositories. Controllers handle HTTP; services handle business logic; repositories handle data access. Always define an interface then a concrete implementation.

ProductService interface

```
public interface ProductService {
ProductResponse getProductById(Integer productId);
Page&lt;ProductResponse&gt; getProducts(Pageable pageable);
List&lt;ProductResponse&gt; searchProductsByName(String keyword);
List&lt;ProductResponse&gt; searchProductsByBrand(Integer brandId);
List&lt;ProductResponse&gt; searchProductsByType(Integer typeId);
List&lt;ProductResponse&gt; searchProductsByBrandandType(Integer brandId, Integer typeId);
List&lt;ProductResponse&gt; searchProductsByBrandTypeAndName(Integer brandId, Integer
typeId, String keyword);
List&lt;BrandResponse&gt; getBrands();
List&lt;TypeResponse&gt; getTypes();
}
```

ProductServiceImpl - key pattern: convert Entity to DTO

```
@Service @Log4j2
public class ProductServiceImpl implements ProductService {
private final ProductRepository productRepository;

@Override
public ProductResponse getProductById(Integer productId) {
Product product = productRepository.findById(productId)
.orElseThrow(() -&gt; new ProductNotFoundException("Product not found"));
return convertToProductResponse(product); // Entity -&gt; DTO
}

@Override
public Page&lt;ProductResponse&gt; getProducts(Pageable pageable) {
return productRepository.findAll(pageable)
.map(this::convertToProductResponse); // map each entity to DTO
}

// Private helper: converts Product entity to ProductResponse DTO
private ProductResponse convertToProductResponse(Product product) {
return ProductResponse.builder()
.id(product.getId()).name(product.getName())
.description(product.getDescription()).price(product.getPrice())
.pictureUrl(product.getPictureUrl())
.productType(product.getType().getName()) // resolve lazy Type
.productBrand(product.getBrand().getName()) // resolve lazy Brand
.build();
}

// BasketService: converts BasketResponse DTO -&gt; Basket entity
// (reverse direction for save operations)
}
```

STEP 9 - REST Controllers - Products, Basket, Auth

Controllers receive HTTP requests and delegate to services. They should contain NO business logic - only HTTP concern mapping.

ProductController.java - /api/products

```
@RestController @RequestMapping("/api/products")
public class ProductController {
// GET /api/products/{id}
@GetMapping("/{id}")
public ResponseEntity&lt;ProductResponse&gt; getProductById(@PathVariable Integer id) { ...
}

// GET /api/products?keyword=&brandId=&typeId=&sort=name&order=asc&page=0&size=10
@GetMapping()
public ResponseEntity&lt;Page&lt;ProductResponse&gt;&gt; getProducts(
@PageableDefault(size=10) Pageable pageable,
@RequestParam(required=false) String keyword,
@RequestParam(required=false) Integer brandId,
@RequestParam(required=false) Integer typeId,
@RequestParam(defaultValue="name") String sort,
@RequestParam(defaultValue="asc") String order) {
// Branching: brandId+typeId+keyword -&gt; searchByBrandTypeAndName()
// brandId+typeId -&gt; searchByBrandAndType()
// keyword only -&gt; searchByName()
// else -&gt; getProducts(pageable) with sort
}
@GetMapping("/brands") public ResponseEntity&lt;List&lt;BrandResponse&gt;&gt; getBrands()
{...}
@GetMapping("/types") public ResponseEntity&lt;List&lt;TypeResponse&gt;&gt; getTypes()
{...}
}
```

BasketController - /api/baskets

```
@RestController @RequestMapping("/api/baskets")
public class BasketController {
@GetMapping // GET all baskets
@GetMapping("/{id}") // GET single basket from Redis
@DeleteMapping("/{id}")// DELETE basket
@PostMapping // POST: create/update basket (POST body = BasketResponse DTO)
}
```

AuthController - /auth

```
@RestController @RequestMapping("/auth")
public class AuthConroller {
@PostMapping("/login")
public ResponseEntity&lt;JwtResponse&gt; login(@RequestBody JwtRequest request) {
authenticate(request.getUsername(), request.getPassword()); // throws on failure
UserDetails userDetails = userDetailsService.loadUserByUsername(request.getUsername());
String token = jwtHelper.generateToken(userDetails);
return ResponseEntity.ok(JwtResponse.builder()
.username(userDetails.getUsername()).token(token).build());
}
@GetMapping("/user") // Decode JWT header and return current user details
```

```
public ResponseEntity&lt;UserDetails&gt; getUserDetails(
@RequestHeader("Authorization") String header) { ... }
}
```

STEP 10 - JWT Security - Helper, Filter and Entry Point

JSON Web Tokens (JWT) are self-contained signed tokens. Once issued, the server verifies the signature on every request without querying the database - fully stateless.

How JWT works in this app

- User POSTs username + password to /auth/login
- Server authenticates, calls JwtHelper.generateToken() and returns a signed token
- Angular stores token in localStorage and attaches it to every API request as: Authorization: Bearer <token>
- JwtAuthenticationFilter intercepts each request, extracts token, validates, sets SecurityContext
- The protected endpoint sees an authenticated user and responds normally

JwtHelper.java - token operations

```
@Component
public class JwtHelper {
public static final long JWT_TOKEN_VALIDITY = 5 * 60 * 60; // 5 hours in seconds
private String secret = "your-256-bit-secret-key"; // Load from env in production!

public String generateToken(UserDetails userDetails) {
Key hmacKey = new SecretKeySpec(secret.getBytes(UTF_8), HS512.getJcaName());
return Jwts.builder()
.setSubject(userDetails.getUsername())
.setIssuedAt(new Date())
.setExpiration(new Date(System.currentTimeMillis() + JWT_TOKEN_VALIDITY * 1000))
.signWith(hmacKey, SignatureAlgorithm.HS512)
.compact();
}

public String getUserNameFromToken(String token) {
return getClaimFromToken(token, Claims::getSubject);
}

public Boolean validateToken(String token, UserDetails userDetails) {
String username = getUserNameFromToken(token);
return username.equals(userDetails.getUsername()) && !isTokenExpired(token);
}
}
```

JwtAuthenticationFilter.java - runs on every request

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {
@Override
protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
FilterChain chain) throws ServletException, IOException {
String header = req.getHeader("Authorization");
if (header != null &amp;amp;&amp;amp; header.startsWith("Bearer")) {
String token = header.substring(7);
String username = jwtHelper.getUserNameFromToken(token);
if (username != null &amp;amp;&amp;amp; SecurityContextHolder.getContext().getAuthentication() ==
null) {
UserDetails ud = userDetailsService.loadUserByUsername(username);
```

```
if (jwtHelper.validateToken(token, ud)) {
UsernamePasswordAuthenticationToken auth =
new UsernamePasswordAuthenticationToken(ud, null, ud.getAuthorities());
auth.setDetails(new WebAuthenticationDetailsSource().buildDetails(req));
SecurityContextHolder.getContext().setAuthentication(auth);
}
}
}
chain.doFilter(req, res); // always continue the filter chain
}
}
```

STEP 11 - Spring Security Config and CORS

SecurityConfig.java - the security policy

```
@Configuration @EnableMethodSecurity
public class SecurityConfig {
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
http
.csrf(AbstractHttpConfigurer::disable) // JWT = stateless, no CSRF needed
.authorizeHttpRequests(r -&gt; r
.requestMatchers("/auth/login").permitAll()
.anyRequest().permitAll())
.exceptionHandling(ex -&gt; ex.authenticationEntryPoint(entryPoint))
.sessionManagement(s -&gt; s.sessionCreationPolicy(STATELESS));
http.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
return http.build();
}
}
```

Note: SessionCreationPolicy.STATELESS is essential. Without it Spring Security creates an HTTP session on the first authenticated request, making the app partially stateful.

CorsConfig.java - allow Angular (port 4200) to call API (port 8080)

```
@Configuration @EnableWebMvc
public class CorsConfig implements WebMvcConfigurer {
@Override
public void addCorsMappings(CorsRegistry registry) {
registry.addMapping("/*/**")
.allowedOrigins("*") // allow any origin (dev only)
.allowedMethods("GET","POST","PUT","DELETE")
.allowedHeaders("*"); // allow Authorization header for JWT
}
}
```

MyConfig.java - in-memory user store

```
@Configuration
public class MyConfig {
@Bean
public UserDetailsService userDetailsService() {
UserDetails user = User.builder()
.username("rahul")
.password(passwordEncoder().encode("Password"))
.roles("admin")
.build();
return new InMemoryUserDetailsManager(user);
}
@Bean
public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
}
```

STEP 12 - Global Exception Handling

```
// ProductNotFoundException.java
public class ProductNotFoundException extends RuntimeException {
public ProductNotFoundException(String message) { super(message); }
}

// CustomExceptionHandler.java
@ControllerAdvice // intercepts exceptions from ANY controller
public class CustomExceptionHandler extends ResponseEntityExceptionHandler {
@ExceptionHandler(ProductNotFoundException.class)
public ResponseEntity&lt;Object&gt; handleProductNotFoundException(
ProductNotFoundException ex, WebRequest request) {
CustomErrorResponse error = new CustomErrorResponse(
HttpStatus.NOT_FOUND, "Product not found", ex.getMessage());
return new ResponseEntity&lt;&gt;(error, HttpStatus.NOT_FOUND);
}
}
```

Note: @ControllerAdvice makes one class handle exceptions from all controllers. Without it you need try-catch in every controller method.

STEP 13 - Angular Project Setup and Folder Structure

Create the Angular project

```
ng new client --routing --style=scss
cd client
npm install bootstrap bootswatch font-awesome ngx-bootstrap@11
npm install ngx-toastr ngx-spinner @paralleldrive/cuid2 xng-breadcrumb
```

Add Bootstrap to angular.json

```
// In angular.json &gt; projects &gt; client &gt; architect &gt; build &gt; options:
"styles": [
"node_modules/bootswatch/dist/flatly/bootstrap.min.css",
"src/styles.scss"
],
```

Feature-based folder structure (lazy-loaded modules)

```
src/app/
+-- app.module.ts # root module
+-- app-routing.module.ts # top-level lazy routes
+-- app.component.* # shell: nav + router-outlet
|
+-- core/ # singleton services and UI chrome
| +-- core.module.ts
| +-- nav-bar/ # top navigation
| +-- guards/ # auth.guard.ts
| +-- interceptors/ # error + loading interceptors
| +-- services/ # loading.service.ts
|
+-- shared/ # reusable components and models
| +-- shared.module.ts
| +-- models/ # product.ts, basket.ts, user.ts ...
| +-- components/ # pagination, order-summary ...
|
+-- home/ # landing page (lazy)
+-- store/ # product list + filters + detail (lazy)
+-- basket/ # shopping cart (lazy)
+-- account/ # login + register (lazy)
+-- checkout/ # address + shipment + review (lazy)
```

STEP 14 - Angular Shared Models - TypeScript Interfaces

TypeScript interfaces define the shape of API response data. They provide compile-time safety and IDE autocomplete.

```
// product.ts
export interface Product {
id: number; name: string; description: string;
price: number; pictureUrl: string;
productType: string; productBrand: string;
}

// basket.ts
export class Basket {
id: string = createId(); // CUID generated client-side
items: BasketItem[] = [];
}
export interface BasketItem {
id: number; name: string; description: string;
price: number; quantity: number;
pictureUrl: string; productBrand: string; productType: string;
}
export interface BasketTotals { shipping: number; subtotal: number; total: number; }

// user.ts
export interface User { username: string; token: string; }

// productData.ts - Spring Page&lt;T&gt; response wrapper
export interface ProductData {
content: Product[]; // products for current page
pageable: Pageable;
totalElements: number; // total matching products
}
export interface Pageable { pageNumber: number; pageSize: number; }
```

STEP 15 - Angular Modules and Lazy Loading

Feature modules are loaded lazily - the browser only downloads the code when the user navigates to that route. This reduces initial bundle size.

app-routing.module.ts - lazy load each feature module

```
const routes: Routes = [
{ path: '', component: HomeComponent },
{ path: 'store', loadChildren: () =&gt;
import('./store/store.module').then(m =&gt; m.StoreModule) },
{ path: 'basket', loadChildren: () =&gt;
import('./basket/basket.module').then(m =&gt; m.BasketModule) },
{ path: 'checkout', loadChildren: () =&gt;
import('./checkout/checkout.module').then(m =&gt; m.CheckoutModule),
canActivate: [AuthGuard] }, // protected - requires login
{ path: 'account', loadChildren: () =&gt;
import('./account/account.module').then(m =&gt; m.AccountModule) },
{ path: 'not-found', component: NotFoundComponent },
{ path: 'server-error', component: ServerErrorComponent },
{ path: '**', redirectTo: 'not-found' },
];
```

app.module.ts - register interceptors

```
@NgModule({
declarations: [AppComponent],
imports: [BrowserModule, AppRoutingModule, BrowserAnimationsModule,
HttpClientModule, CoreModule, HomeModule,
NgxSpinnerModule.forRoot({ type: 'square-jelly-box' })],
providers: [
{ provide: HTTP_INTERCEPTORS, useClass: ErrorInterceptor, multi: true },
{ provide: HTTP_INTERCEPTORS, useClass: LoadingInterceptor, multi: true },
],
bootstrap: [AppComponent]
})
export class AppModule {}
```

STEP 16 - StoreService and AccountService - HTTP Calls

StoreService - fetches products, brands and types

```
@Injectable({ providedIn: 'root' })
export class StoreService {
public apiUrl = 'http://localhost:8080/api/products';
constructor(private http: HttpClient) {}

// GET /api/products?brandId=&typeId=&keyword=&page=&size=
getProducts(brandId?: number, typeId?: number, url?: string): Observable&lt;ProductData&gt;
{
const finalUrl = url || this.apiUrl;
return this.http.get&lt;ProductData&gt;(finalUrl);
}
getProduct(id: number): Observable&lt;Product&gt; {
return this.http.get&lt;Product&gt;(`${this.apiUrl}/${id}`);
}
getBrands(): Observable&lt;Brand[]&gt; {
return this.http.get&lt;Brand[]&gt;(`${this.apiUrl}/brands`);
}
getTypes(): Observable&lt;Brand[]&gt; {
return this.http.get&lt;Brand[]&gt;(`${this.apiUrl}/types`);
}
}
```

AccountService - login, logout, JWT storage

```
@Injectable({ providedIn: 'root' })
export class AccountService {
apiUrl = 'http://localhost:8080/auth';
private currentUserSource = new BehaviorSubject&lt;User | null&gt;(null);
currentUser$ = this.currentUserSource.asObservable();

login(values: any): Observable&lt;void&gt; {
return this.http.post&lt;User&gt;(this.apiUrl + '/login', values).pipe(
map(user =&gt; {
localStorage.setItem('token', user.token);
this.currentUserSource.next(user);
})
);
}
logout(): void {
localStorage.removeItem('token');
this.currentUserSource.next(null);
this.router.navigateByUrl('/');
}
isAuthenticated(): boolean { return !!localStorage.getItem('token'); }
loadUser(): void {
const token = localStorage.getItem('token');
if (token) {
const headers = new HttpHeaders({ Authorization: `Bearer ${token}` });
this.http.get&lt;User&gt;(`${this.apiUrl}/user`, { headers })
.subscribe(user =&gt; this.currentUserSource.next(user));
}
}
}
```

STEP 17 - BasketService - Reactive State with BehaviorSubject

BasketService manages the cart client-side using **BehaviorSubject** - a reactive stream that always holds the latest value and emits it to any new subscriber immediately.

```
@Injectable({ providedIn: 'root' })
export class BasketService {
apiUrl = 'http://localhost:8080/api/baskets';
private basketSource = new BehaviorSubject&lt;Basket | null&gt;(null);
basketSource$ = this.basketSource.asObservable();
private basketTotalSource = new BehaviorSubject&lt;BasketTotals&gt;({
subtotal:0,shipping:0,total:0 });
basketTotalSource$ = this.basketTotalSource.asObservable();

constructor(private http: HttpClient) {
const stored = localStorage.getItem('basket');
if (stored) { this.basketSource.next(JSON.parse(stored)); this.calculateTotals(); }
}

addItemToBasket(item: Product, quantity = 1): void {
const itemToAdd = this.mapProductToBasket(item);
const basket = this.getCurrentBasket() ?? this.createBasket();
basket.items = this.upsertItems(basket.items, itemToAdd, quantity);
this.setBasket(basket);
}

upsertItems(items: BasketItem[], itemToAdd: BasketItem, qty: number): BasketItem[] {
const existing = items.find(x =&gt; x.id === itemToAdd.id);
if (existing) { existing.quantity += qty; }
else { itemToAdd.quantity = qty; items.push(itemToAdd); }
return items;
}

setBasket(basket: Basket): void {
this.http.post&lt;Basket&gt;(this.apiUrl, basket).subscribe(b =&gt; {
this.basketSource.next(b);
this.calculateTotals();
localStorage.setItem('basket', JSON.stringify(b));
});
}

remove(itemId: number): void {
const basket = this.getCurrentBasket();
if (!basket) return;
basket.items.splice(basket.items.findIndex(p =&gt; p.id === itemId), 1);
if (basket.items.length === 0) {
this.basketSource.next(null); localStorage.removeItem('basket');
} else { this.setBasket(basket); }
}

private calculateTotals(): void {
const basket = this.getCurrentBasket();
if (!basket) return;
const subtotal = basket.items.reduce((acc, i) =&gt; (i.price * i.quantity) + acc, 0);
this.basketTotalSource.next({ shipping: 0, subtotal, total: subtotal });
}
}
```

STEP 18 - HTTP Interceptors - Error Handling and Loading Spinner

Interceptors sit in the request/response pipeline. They add cross-cutting behaviour (error handling, loading spinners) without modifying every individual service.

ErrorInterceptor - handle 404 and 500 globally

```
@Injectable()
export class ErrorInterceptor implements HttpInterceptor {
constructor(private router: Router, private toastr: ToastrService) {}

intercept(request: HttpRequest&lt;unknown&gt;, next: HttpHandler):
Observable&lt;HttpEvent&lt;unknown&gt;&gt; {
return next.handle(request).pipe(
catchError(error =&gt; {
if (error.status === 404) {
this.toastr.error('Resource not found');
this.router.navigate(['/not-found']);
} else if (error.status === 500) {
this.toastr.error('Server error occurred');
this.router.navigate(['/server-error']);
}
throw error;
})
);
}
}
```

LoadingInterceptor - show/hide spinner

```
@Injectable()
export class LoadingInterceptor implements HttpInterceptor {
constructor(private loadingService: LoadingService) {}

intercept(request: HttpRequest&lt;unknown&gt;, next: HttpHandler):
Observable&lt;HttpEvent&lt;unknown&gt;&gt; {
this.loadingService.loading();
return next.handle(request).pipe(
delay(1000), // remove in production
finalize(() =&gt; this.loadingService.idle())
);
}
}

// LoadingService.ts
@Injectable({ providedIn: 'root' })
export class LoadingService {
constructor(private spinner: NgxSpinnerService) {}
loading(): void { this.spinner.show(); }
idle(): void { this.spinner.hide(); }
}
```

STEP 19 - Store Component - Filtering, Sorting and Pagination

The Store component builds a dynamic query URL from the user's current filter/sort/page selections and calls StoreService to hit the Spring Boot API.

```
export class StoreComponent implements OnInit {
ngOnInit(): void {
this.storeData.selectedBrand = { id: 0, name: 'All' };
this.storeData.selectedType = { id: 0, name: 'All' };
this.fetchProducts();
this.getBrands();
this.getTypes();
}

fetchProducts(page = 1): void {
const brandId = this.storeData.selectedBrand?.id;
const typeId = this.storeData.selectedType?.id;
let url = `${this.storeService.apiUrl}?`;
if (brandId &amp;&amp; brandId !== 0) url += `brandId=${brandId}&amp;`;
if (typeId &amp;&amp; typeId !== 0) url += `typeId=${typeId}&amp;`;
if (this.storeData.search) url += `keyword=${this.storeData.search}&amp;`;
url += `page=${page - 1}&amp;size=${this.storeData.pageSize}`;
if (this.storeData.selectedSort !== 'asc') url += `&amp;sort=name&amp;order=desc`;

this.storeService.getProducts(brandId, typeId, url).subscribe({
next: data =&gt; {
this.storeData.products = data.content;
this.storeData.totalElements = data.totalElements;
this.storeData.currentPage = data.pageable.pageNumber + 1;
}
});
}

selectBrand(brand: Brand): void { this.storeData.selectedBrand = brand;
this.fetchProducts(); }
selectType(type: Type): void { this.storeData.selectedType = type; this.fetchProducts(); }
onSortChange(): void { this.fetchProducts(); }
onSearch(): void { this.fetchProducts(); }
onReset(): void { this.storeData.search = ''; this.fetchProducts(); }
pageChanged(event: PageChangedEvent): void {
if (event.page !== this.storeData.currentPage) {
this.fetchProducts(event.page);
}
}
}
```

STEP 20 - Auth Guard and Running the Full App

Auth Guard - route protection

```
@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
constructor(private accountService: AccountService, private router: Router) {}

canActivate(route: ActivatedRouteSnapshot, state: RouterStateSnapshot): boolean {
if (this.accountService.isAuthenticated()) { return true; }
this.accountService.redirectUrl = state.url;
this.router.navigate(['/account/login']);
return false;
}
}
```

1. Start Docker containers

```
cd docker/
docker compose up -d
docker ps # verify both are Up
```

2. Start Spring Boot backend

```
./mvnw spring-boot:run # API at http://localhost:8080
```

3. Start Angular frontend

```
cd client/
npm install
ng serve --open # opens http://localhost:4200
```

4. Test the API

Endpoint	Expected Result
GET /api/products	Paginated JSON of all products
GET /api/products/1	Single product JSON
GET /api/products?brandId=1	Products filtered by brand
GET /api/products/brands	All brands list
POST /auth/login {username,pw}	JWT token in response
GET /api/baskets/{id}	Basket from Redis
POST /api/baskets	Create/update basket

Default login credentials

```
Username: rahul
Password: Password
# Defined in MyConfig.java
```

Common issues

- **CORS error in browser:** Check CorsConfig.java has allowedOrigins set correctly
- **401 Unauthorized:** JWT expired or missing - re-login to get a fresh token
- **Port 3306 refused:** Docker MySQL not running - run docker compose up -d
- **Redis connection error:** Docker Redis not running - check docker ps
- **Products page empty:** data.sql may not have seeded - check docker logs

---

You have now built a complete fullstack e-commerce application. The architecture scales naturally: swap InMemoryUserDetailsManager for a JPA-backed user store, add Stripe payments, deploy the backend on Cloud Run and the frontend on Firebase Hosting for a production app.